```julia
begin
    using LinearAlgebra
    using SparseArrays
    using PlutoUI
    using PlutoTeachingTools
    using HypertextLiteral: @htl, @htl_str
end
```

Table of Contents

Direct methods for linear systems

- What Julia's `\` does under the hood
- Solving triangular systems
- Gaussian elimination and LU factorisation
- Gaussian eliminitation with pivoting

⚠ TODO ⚠

- Memory usage and fill-in
  - Sparse matrices
  - LU factorisation of sparse matrices
- Computational cost of LU factorisation
  - Scalar product
  - Matrix-vector product
  - LU factorisation
- Numerical stability
  - Matrix norm and condition numbers
  - Stability result

Direct methods for linear systems

In the previous chapter on polynomial interpolation we were already confronted with the need to solve linear systems, that is a system of equations of the form

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \tag{1}$$

where we are given a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ as well as a right-hand side $\mathbf{b} \in \mathbb{R}^n$. As the solution we seek the unknown $\mathbf{x} \in \mathbb{R}^n$.

Solving such equations is a standard exercise in linear algbra and you probably have already done it in previous courses. However, you might also know that solving such linear systems is not always equally easy.

This one, for example, is particularly difficult:

```
M_difficult = 2×2 Matrix{Float64}:
              3.14159  2.71828
              3.14159  2.71828
```

```
M_difficult = [π          e;
               355/113    23225/8544]   # Rational approximations to π and e
```

```
b_difficult = [1.0, 1.0]
```

```
b_difficult = [1.0, 1.0]
```

Solving it in 64-bit `Float64` gives this result:

```
[-0.00958449, 0.378957]
```

```
M_difficult \ b_difficult
```

while using 32-bit `Float32` gives this:

```
[-0.114318, 0.5]
```

```
let
    M_32 = convert(Matrix{Float32}, M_difficult)
    b_32 = convert(Vector{Float32}, b_difficult)
    M_32 \ b_32
end
```

These clearly differ and we would probably trust the `Float64` result more. But is this justified ? And what are conditions to quantify that a linear system is more challenging than another ?

This is what we want to explore in this part of the course.

What Julia's \ does under the hood

For solving linear systems, we so far employed Julia's backslash `\` operator. So let's find out what it actually does under the hood. We take the problem

```julia
begin
    A = Float64[-4  3 -1;
                 2  1  0;
                 4 -3  4]
    b = [2, 4, -2]
end;
```

as an example. Normally we would now just do `A \ b`, so let's check what this calls

```
# 1 method for generic function \ from Base:
```

- \(A::**AbstractMatrix**, B::**AbstractVecOrMat**) in LinearAlgebra at /opt/hostedtoolcache/julia/1.11.3/x64/share/julia/stdlib/v1.11/LinearAlgebra/src/generic.jl:1118

```julia
methods(\, (Matrix, Vector))
```

Ok, so this calls into Julia's linear algebra library. The code is located here. Essentially it performs an **LU factorisation**:

```
LU = LinearAlgebra.LU{Float64, Matrix{Float64}, Vector{Int64}}
     L factor:
     3×3 Matrix{Float64}:
       1.0  0.0  0.0
      -0.5  1.0  0.0
      -1.0  0.0  1.0
     U factor:
     3×3 Matrix{Float64}:
      -4.0  3.0  -1.0
       0.0  2.5  -0.5
       0.0  0.0   3.0
```

```julia
LU = lu(A)
```

which as we can see *factorises* the matrix $\mathbf{A}$ into a **lower triangular** matrix $\mathbf{L}$ and an **upper triangular** matrix $\mathbf{U}$. More formally:

Definition: LU factorisation

Given a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ find a lower triangular matrix $\mathbf{L} \in \mathbb{R}^{n \times n}$ and an upper triangular matrix $\mathbf{U} \in \mathbb{R}^{n \times n}$, such that

$$\mathbf{A} = \mathbf{L}\mathbf{U} \tag{2}$$

We can easily check that by multiplying $\mathbf{L}$ and $\mathbf{U}$ we indeed revover $\mathbf{A}$:

```
3×3 Matrix{Float64}:
 -4.0   3.0  -1.0
  2.0   1.0   0.0
  4.0  -3.0   4.0
```

```
LU.L * LU.U
```

```
3×3 Matrix{Float64}:
 0.0  0.0  0.0
 0.0  0.0  0.0
 0.0  0.0  0.0
```

```
A - LU.L * LU.U
```

Now we are a step closer to what happens, but why is this useful ?

Solving triangular systems

The factorisation into two triangular matrices is useful, because it is especially easy to solve a system where the matrix is triangular. For example consider the *lower* triangular system

$$\begin{pmatrix} 4 & 0 & 0 & 0 \\ 3 & -1 & 0 & 0 \\ -1 & 0 & 3 & 0 \\ 1 & -1 & -1 & 2 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 8 \\ 5 \\ 0 \\ 1 \end{pmatrix}.$$

The first row of this system simply states $4x_1 = 8$, which is very easy to solve, namely $x_1 = 8/4 = 2$. The second row states $3x_1 - x_2 = 5$. However, x_1 is already known and can be inserted to find $x_2 = -(5 - 3 \cdot 2) = 1$. Following the same idea the third row gives $x_3 = (0 + 1 \cdot 2)/3 = 2/3$ and the last row $x_4 = (1 - 1 \cdot 2 + 1 \cdot 1 + 1 \cdot 2/3)/2 = 1/3$. In total we found

$$\mathbf{x} = \begin{pmatrix} 2 \\ 1 \\ 2/3 \\ 1/3 \end{pmatrix}.$$

Generalising to an arbitrary 4x4 lower-rectangular matrix

$$\begin{pmatrix} L_{11} & 0 & 0 & 0 \\ L_{21} & L_{22} & 0 & 0 \\ L_{31} & L_{32} & L_{33} & 0 \\ L_{41} & L_{42} & L_{43} & L_{44} \end{pmatrix} \mathbf{x} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}$$

this solution algorithm can be expressed as

$$\begin{aligned} x_1 &= \frac{b_1}{L_{11}}, \\ x_2 &= \frac{b_2 - L_{21}x_1}{L_{22}}, \\ x_3 &= \frac{b_3 - L_{31}x_1 - L_{32}x_2}{L_{33}}, \\ x_4 &= \frac{b_4 - L_{41}x_1 - L_{42}x_2 - L_{43}x_3}{L_{44}}. \end{aligned} \tag{3}$$

We obtain

Algorithm 1: Forward substitution

Given a lower-triangular matrix $\mathbf{L} \in \mathbb{R}^{n \times n}$ a linear system $\mathbf{Lx} = \mathbf{b}$ can be solved by looping from $i = 1, \ldots n$ and computing

$$x_i = \frac{1}{L_{ii}} \left(b_i - \sum_{j=1}^{i-1} L_{ij} x_j \right)$$

or in form of an implementation

```
forward_substitution (generic function with 1 method)
```

```julia
function forward_substitution(L, b)
    n = size(L, 1)
    x = zeros(n)
    x[1] = b[1] / L[1, 1]
    for i in 2:n
        row_sum = sum(L[i, j] * x[j] for j in 1:i-1)
        x[i] = (b[i] - row_sum) / L[i, i]
    end
    x
end
```

For upper triangular matrices we solve linear systems proceeds following the same idea — just in this case we start from the last row and not the first. For example a system

$$\begin{pmatrix} U_{11} & U_{12} & U_{13} & U_{14} \\ 0 & U_{22} & U_{23} & U_{24} \\ 0 & 0 & U_{33} & U_{34} \\ 0 & 0 & 0 & U_{44} \end{pmatrix} \mathbf{x} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}$$

we solve by starting with x_4 and working our way forward to x_1:

$$\begin{aligned} x_4 &= \frac{b_4}{U_{44}}, \\ x_3 &= \frac{b_3 - U_{34}x_4}{U_{33}}, \\ x_2 &= \frac{b_2 - U_{23}x_3 - U_{24}x_4}{U_{22}}, \\ x_1 &= \frac{b_1 - U_{12}x_2 - U_{13}x_3 - U_{14}x_4}{U_{11}}. \end{aligned} \tag{4}$$

More formally this algorithm is called

Algorithm 2: Backward substitution

Given an upper-triangular matrix $\mathbf{U} \in \mathbb{R}^{n \times n}$ a linear system $\mathbf{U}\mathbf{x} = \mathbf{b}$ can be solved by looping in reverse order $i = n, \ldots, 1$ and computing

$$x_i = \frac{1}{U_{ii}} \left(b_i - \sum_{j=i+1}^{n} U_{ij} x_j \right)$$

or in code

```
backward_substitution (generic function with 1 method)
```

```julia
function backward_substitution(U, b)
    n = size(L, 1)
    x = zeros(n)
    x[n] = b[n] / U[n, n]
    for i in n-1:-1:1
        row_sum = sum(U[i, j] * x[j] for j in i+1:n)
        x[i] = (b[i] - row_sum) / U[i, i]
    end
    x
end
```

Based on our discussion we now understand why it is advantageous to perform an LU factorisation when solving a linear system: For both the resulting triangular matrix $\mathbf{L}$ as well as the matrix $\mathbf{U}$, simple solution algorithms are available. In summary we thus obtain

Algorithm 3: Solving linear systems by LU factorisation / Gaussian elimination

We are given a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, a right-hand side $\mathbf{b} \in \mathbb{R}^n$. This algorithm computes the solution $\mathbf{x} \in \mathbb{R}^n$ to $\mathbf{A}\mathbf{x} = \mathbf{b}$:

1. Factorise $\mathbf{A} = \mathbf{L}\mathbf{U}$.
2. Solve $\mathbf{L}\mathbf{z} = \mathbf{b}$ for $\mathbf{z}$ using *forward* substitution.
3. Solve $\mathbf{U}\mathbf{x} = \mathbf{z}$ for $\mathbf{x}$ using *backward* substitution.

A few remarks are in order:

- We have so far not discussed how to even compute the LU factorisation. As we will see in the next section this step will actually be accomplished by the Gaussian elimination algorithm, which probably already know from your linear algebra lectures.
- A severe drawback of this naive algorithm is immediately apparent from our 4x4 example problem, where (3) and (4) provide explicit solution expressions. If by any chance an element L_{ii} or U_{ii} happens to be zero, the algorithm cannot work. This already points to a problem we will discuss in the section on *pivoting*.
- Note, that steps 2. and 3. of the algorithm do *not depend* on the original matrix $\mathbf{A}$, but only on the right hand side $\mathbf{b}$. In other words as soon as the factorisation $\mathbf{A} = \mathbf{L}\mathbf{U}$ has been computed, solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ for many right hand sides $\mathbf{b}$ only requires the execution of steps 2. and 3. As we will see later, step 1. is the most expensive step. Therefore once $\mathbf{A}$ has been factorised, the cost for solving an *arbitrary* linear systems involving $\mathbf{A}$ is reduced as its factorised form $\mathbf{L}\mathbf{U}$ can be used in its place.

Gaussian elimination and LU factorisation

Final question and the missing puzzle piece is how to compute LU factorization. As it turns out the Gaussian elimination algorithm you already learned in previous linear algebra classes provides exactly this missing ingredient.

Recall that **Gaussian elimination** allows us to transform a linear system into an upper triangular system using elementary operations, which combine the different equations.

Example: Manual Gaussian elimination

We will discuss one example, namely the linear system

$$\underbrace{\begin{pmatrix} 2 & 1 & 0 \\ -4 & 3 & -1 \\ 4 & -3 & 4 \end{pmatrix}}_{=\mathbf{A}} \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_{=\mathbf{x}} = \underbrace{\begin{pmatrix} 4 \\ 2 \\ -2 \end{pmatrix}}_{=\mathbf{b}}.$$

Let us call $r_1^{(1)}$, $r_2^{(1)}$ and $r_3^{(1)}$ the three equations of the system:

$$\begin{array}{lrrrl} r_1^{(1)}: & 2x_1 & +x_2 & & =4 \\ r_2^{(1)}: & -4x_1 & +3x_2 & -x_3 & =2 \\ r_3^{(1)}: & 4x_1 & -3x_2 & +4x_3 & =-2 \end{array}$$

on which we perform the following operations.

1st step: Zero out all but the first row in the first column.

$$\begin{array}{lllrrrl} r_1^{(2)} \leftarrow r_1^{(1)} & & \Longrightarrow & 2x_1 & +x_2 & & =4 \\ r_2^{(2)} \leftarrow r_2^{(1)} - \underbrace{\left(\frac{-4}{2}\right)}_{L_{21}} r_1^{(1)} & & \Longrightarrow & & 5x_2 & -x_3 & =10 \\ r_3^{(2)} \leftarrow r_3^{(1)} - \underbrace{\left(\frac{4}{2}\right)}_{L_{31}} r_1^{(1)} & & \Longrightarrow & & -5x_2 & +4x_3 & =-10. \end{array}$$

2nd step: Zero out all but the first and second row in the second column.

$$\begin{array}{lllrrrl} r_1^{(3)} \leftarrow r_1^{(2)} & & \Longrightarrow & 2x_1 & +x_2 & & =4 \\ r_2^{(3)} \leftarrow r_2^{(2)} & & \Longrightarrow & & 5x_2 & -x_3 & =10 \\ r_3^{(3)} \leftarrow r_3^{(2)} - \underbrace{\left(\frac{-5}{5}\right)}_{L_{32}} r_2^{(2)} & & \Longrightarrow & & & 3x_3 & =0. \end{array}$$

After this step we have thus obtained an upper-triangular system, which we would solve using backward substitution. Note that during the iterations the system matrix got slowly reduced to an upper triangular matrix, which we can identify with $\mathbf{U}$:

$$\underbrace{\mathbf{A}^{(1)}}_{=\mathbf{A}} = \begin{pmatrix} 2 & 1 & 0 \\ -4 & 3 & -1 \\ 4 & -3 & 4 \end{pmatrix} \Rightarrow \mathbf{A}^{(2)} = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 5 & -1 \\ 0 & -5 & 4 \end{pmatrix} \Rightarrow \mathbf{A}^{(3)} = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 5 & -1 \\ 0 & 0 & 3 \end{pmatrix} =$$

In turn the matrix $\mathbf{L}$ is obtained by collecting the factors we employed to multiply the rows. We add diagonal entries of 1 to indicate that the i-th row has been left unchanged in the i-th step. Accumulating step by step we obtain

$$\mathbf{L}^{(1)} = \begin{pmatrix} 1 & & \\ -\frac{4}{2} & & \\ \frac{4}{2} & & \end{pmatrix} \Rightarrow \mathbf{L}^{(2)} = \begin{pmatrix} 1 & & \\ -2 & 1 & \\ 2 & -\frac{5}{5} & \end{pmatrix} \Rightarrow \mathbf{L}^{(3)} = \begin{pmatrix} 1 & & \\ -2 & 1 & \\ 2 & -1 & 1 \end{pmatrix} = \mathbf{L}$$

Finally by multiplying out the matrices we easily verify

$$\mathbf{LU} = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 2 & -1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 0 \\ 0 & 5 & -1 \\ 0 & 0 & 3 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 0 \\ -4 & 3 & -1 \\ 4 & -3 & 4 \end{pmatrix} = \mathbf{A}$$

So effectively Gaussian elimination provides us with an approach to compute the LU factorisation. As a result Algorithm 3 is nothing but a formalised form of the procedure you used in your linear algebra lectures to solve linear systems using Gaussian elimination. We summarise:

Algorithm 4: LU factorisation

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$

Output: $\mathbf{U} \in \mathbb{R}^{n \times n}$, $\mathbf{L} \in \mathbb{R}^{n \times n}$

- $\mathbf{A}^{(1)} = \mathbf{A}$
- for $k = 1, \ldots, n-1$ *(algorithm steps)*
  - $L_{kk} = 1$
  - for $i = k+1, \ldots, n$ *(Loop over rows)*
    - $L_{ik} = \frac{A_{ik}^{(k)}}{A_{kk}^{(k)}}$
    - for $j = k+1, \ldots n$ *(Loop over columns)*
      - $A_{ij}^{(k+1)} = A_{ij}^{(k)} - L_{ik} A_{kj}^{(k)}$
- $\mathbf{U} = \mathbf{A}^{(n)}$

An implementation of LU factorisation could be realised as such:

```
factorise_lu (generic function with 1 method)
```

```julia
function factorise_lu(A)
    n = size(A, 1)
    L = diagm(ones(n))   # Initialise L by ones on diagonal, zeros elsewhere
    U = zeros(n, n)
    Aᵏ = float(copy(A))  # Make a copy of A and ensure that all entries
                         # are converted to floating-point numbers

    for k in 1:n-1           # Algorithm steps
        U[k, :] = Aᵏ[k, :]  # Copy k-th row to U, since this row is in its final form.
                             # From now on in the algorithm use U[k, :] instead of Aᵏ,
                             # since Aᵏ is updated in-place to become Aᵏ⁺¹
        for i in k+1:n       # Loop over rows
            L[i, k] = Aᵏ[i, k] / U[k, k]
            for j = k+1:n    # Loop over columns
                Aᵏ[i, j] = Aᵏ[i, j] - L[i, k] * U[k, j]  # Update Aᵏ in-place
            end
        end
    end
    U[n, n] = Aᵏ[n, n]

    # Return L and U using Julia datastructures to indicate
    # their special lower-triangular and upper-triangular form.
    return LowerTriangular(L), UpperTriangular(U)
end
```

Let us test our algorithms `factorise_lu`, `forward_substitution` and `backward_substitution` on our earlier example problem $\mathbf{A}\mathbf{x} = \mathbf{b}$ with

```
3×3 Matrix{Float64}:
 -4.0   3.0  -1.0
  2.0   1.0   0.0
  4.0  -3.0   4.0
```

```julia
A
```

```
▸[2, 4, -2]
```

```julia
b
```

We follow Algorithm 3. First we need to find the LU factorisation:

```
▸(3×3 LowerTriangular{Float64, Matrix{Float64}}:, 3×3 UpperTriangular{Float64, Matrix{
    1.0   ⋅    ⋅                                  -4.0  3.0  -1.0
```

```julia
L, U = factorise_lu(A)
```

Next we forward substitute:

```
z = ▸[2.0, 5.0, 0.0]
```

```
z = forward_substitution(L, b)
```

Finally we backward substitute:

```
x = ▸[1.0, 2.0, 0.0]
```

```
x = backward_substitution(U, z)
```

and check the result:

```
▸[0.0, 0.0, 0.0]
```

```
A * x - b
```

Hooray ! This suceeded. But does it always work ? The answer to this is unfortunately now as we will see in the next section.

Gaussian eliminitation with pivoting

Consider the matrix

```
AA = 3×3 Matrix{Int64}:
     1  2  3
     2  4  5
     7  8  9
```

```
AA = [1 2 3;
      2 4 5;
      7 8 9]
```

the LU factorisation gives strange results

```
3×3 LowerTriangular{Float64, Matrix{Float64}}:
 1.0     ⋅    ⋅
 2.0    1.0   ⋅
 7.0  -Inf   1.0
```

```
let
    L, U = factorise_lu(AA)
    L
end
```

The source of the problem is easily found. After the first step of the algorithm we get the matrices

$$\mathbf{A}^{(2)} = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & -1 \\ 0 & -6 & -12 \end{pmatrix}, \qquad \mathbf{L}^{(1)} = \begin{pmatrix} 1 & & \\ 2 & & \\ 7 & & \end{pmatrix}.$$

As a result the element `A^k[k, k]` is zero and if we blindly continue in step `k = 2` a division by zero is encountered in the computation of `L[k, k]` — which reflects above in the introduction of a `-Inf` in the matrix. From this point our computations are numerical garbage. Due their central role in the Gaussian elimination algorithm the elements $\left(A^{(k)}\right)_{kk}$ — respectively `A^k[k, k]` in the implementation — are usually referred to as **pivots**.

To overcome the encountered problem we will consider a strategy called **row pivoting**, i.e. where we allow ourselves some flexibility by selecting the pivot amongst the $n-k$ lowermost columns in the k-th step of Gaussian elimination. This is usually performed by *permuting* (swapping the order) of the $n-k$ lowermost columns.

Looking at $\mathbf{A}^{(2)}$ it seems very reasonable to just swap the second and the third column in $\mathbf{A}^{(2)}$ and thus move the -6 to become the new pivot. For consistency we not only need to swap $A^{(2)}$, but also $\mathbf{L}^{(2)}$. This gives us

$$\widetilde{\mathbf{A}}^{(2)} = \begin{pmatrix} 1 & 2 & 3 \\ 0 & -6 & -12 \\ 0 & 0 & -1 \end{pmatrix}, \qquad \widetilde{\mathbf{L}}^{(1)} = \begin{pmatrix} 1 & & \\ 7 & & \\ 2 & & \end{pmatrix}.$$

If we continue the algorithm now, the -6 sits in the position of the pivot $\left(A^{(k)}\right)_{kk}$ and the division by zero is avoided.

In fact in this particular case there is nothing left to do and the matrix $\widetilde{\mathbf{A}}^{(2)}$ already is in upper triangular form. We obtain from our algorithm

$$\mathbf{U} = \mathbf{A}^{(3)} = \widetilde{\mathbf{A}}^{(2)} = \begin{pmatrix} 1 & 2 & 3 \\ 0 & -6 & -12 \\ 0 & 0 & -1 \end{pmatrix} \qquad \mathbf{L} = \mathbf{L}^{(3)} = \begin{pmatrix} 1 & 0 & 0 \\ 7 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix}.$$

Since the additional row permutation we performed not taken into account in the $\mathbf{L}$ and $\mathbf{U}$ factors, multiplying them out will not yield $\mathbf{A}$, but a matrix where the second and third rows of $\mathbf{A}$ are swapped:

$$\mathbf{LU} = \begin{pmatrix} 1 & 0 & 0 \\ 7 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 0 & -6 & -12 \\ 0 & 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 7 & 8 & 9 \\ 2 & 4 & 5 \end{pmatrix}$$

To resolve this, we introduce the **permutation matrix**

$$\mathbf{P} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix},$$

which is obtained by performing exactly the same row permutations we did during the algorithm to the identity matrix — i.e. we swap the second and third row. With this matrix we can easily check that

$$\mathbf{LU} = \mathbf{PA}.$$

If we thus extend our `factorise_lu` function to additionally perform such pivoting permutations, the Gaussian elimination algorithm would terminate successfully. But pivoting brings additional opportunities. As it turns out numerical stability of LU factorisation improves if one permutes not only if a pivot $\left(A^{(k)}\right)_{kk}$ is zero, but in general if one *always* swaps the order of the rows such that the pivot is taken as large as possible. In other words in the k-th step of LU factorisation we always exchange row k with the row l where l satisfies

$$\left|\left(A^{(k)}\right)_{lk}\right| \leq \left|\left(A^{(k)}\right)_{ik}\right| \qquad \text{for all } i = k, \ldots n.$$

The appropriate swaps are tracked and returned as well. Note that instead of returning a permutation matrix $\mathbf{P}$ (which requries to store n^2 elements) it is usually more convenient to store a permutation vector $\mathbf{p}$, which has only n elements. This vector tracks the indices of the rows of $\mathbf{A}$ in the order they are used as pivots. In other words if

```
idmx = 4×4 Matrix{Float64}:
        1.0  0.0  0.0  0.0
        0.0  1.0  0.0  0.0
        0.0  0.0  1.0  0.0
        0.0  0.0  0.0  1.0
```

```
idmx = diagm(ones(4))
```

is the identity matrix and

```
perm = [1, 3, 2, 4]
```

```
perm = [1, 3, 2, 4]
```

the permutation vector, then

```
4×4 Matrix{Float64}:
 1.0  0.0  0.0  0.0
 0.0  0.0  1.0  0.0
 0.0  1.0  0.0  0.0
 0.0  0.0  0.0  1.0
```

```julia
idmx[perm, :]
```

is the permutation matrix.

The code below presents an implementation of row-pivoted LU factorisation.

```
factorise_lu_pivot (generic function with 1 method)
```

```julia
function factorise_lu_pivot(A)
    n = size(A, 1)
    L = zeros(n, n)
    U = zeros(n, n)
    p = fill(0, n)
    Aᵏ = float(copy(A))  # Make a copy of A and ensure that all entries
                         # are converted to floating-point numbers

    for k in 1:n-1             # Algorithm steps
        p[k] = argmax(abs.(Aᵏ[:, k]))  # Find row with maximal pivot

        U[k, :] = Aᵏ[p[k], :]  # Copy pivot row to U, use U now instead of Aᵏ,
                               # which is again updated in-place
        for i in 1:n           # Row loop: Note the full range as any row may
                               #           be non-zero
            L[i, k] = Aᵏ[i, k] / U[k, k]
            for j = 1:n        # Column loop: Again full range
                Aᵏ[i, j] = Aᵏ[i, j] - L[i, k] * U[k, j]
            end
        end
    end
    p[n] = argmax(abs.(Aᵏ[:, n]))
    U[n, n] = Aᵏ[p[n], n]
    L[:, n] = Aᵏ[:, n] / U[n, n]

    # To simplify assembling L we so far kept the rows in the same order
    # as in A. To make the matrix upper triangular we also apply the column
    # permutation p before returning the results.
    LowerTriangular(L[p, :]), UpperTriangular(U), p
end
```

We conclude with a theoretical result:

Theorem 1

Every non-singular matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ admits a factorisation

$$\mathbf{PA} = \mathbf{LU}$$

where $\mathbf{L}$ is lower-triangular, $\mathbf{U}$ upper-triangular and $\mathbf{P}$ is a permutation matrix.

Note, that this implies that `factorise_lu_pivot` can be employed to construct a robust method for solving linear systems, namely Gaussian elimination with pivoting. The main idea is that using pivoted LU factorisation we can write:

$$\mathbf{Ax} = \mathbf{b} \quad \Longleftrightarrow \quad \mathbf{PAx} = \mathbf{Pb} \quad \Longleftrightarrow \quad \mathbf{LUx} = \mathbf{Pb}$$

Algorithm 5: Solving linear systems with pivoted LU factorisation

Given a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, a right-hand side $\mathbf{b} \in \mathbb{R}^n$ the linear system $\mathbf{Ax} = \mathbf{b}$ can be solved for $\mathbf{x} \in \mathbb{R}^n$ as follows:

1. Factorise $\mathbf{PA} = \mathbf{LU}$.
2. Solve $\mathbf{Lz} = \mathbf{Pb}$ for $\mathbf{z}$ using *forward* substitution.
3. Solve $\mathbf{Ux} = \mathbf{z}$ for $\mathbf{x}$ using *backward* substitution.

To conclude we return to our starting question: Understanding the `\` operator in Julia.

- We already saw that julia essentially solves `A \ b` by replacing it by `lu(A) \ b`.
- Julia's `lu` by default also performs a row-pivoted LU factorisation.
- The returned object (an `LinearAlgebra.LU` object) internally stores both the `L`, the `U` and the permutation vector `p`
- When we use `\` with an `LU` object it, Julia immediately exploits the factorised form and performs first (permuted) forward substitution (step 2.) then backward substitution (step 3.).
- Therefore overall `A \ b` exactly performs Algorithm 5 above.

We can check against our implementation:

```
fac = LinearAlgebra.LU{Float64, Matrix{Float64}, Vector{Int64}}
      L factor:
      3×3 Matrix{Float64}:
       1.0       0.0  0.0
       0.285714  1.0  0.0
       0.142857  0.5  1.0
      U factor:
      3×3 Matrix{Float64}:
       7.0  8.0      9.0
       0.0  1.71429  2.42857
       0.0  0.0      0.5
```

```
fac = lu(AA)
```

```
▸[3, 2, 1]
```

```
fac.p
```

```
3×3 Matrix{Float64}:
 1.0       0.0  0.0
 0.285714  1.0  0.0
 0.142857  0.5  1.0
```

```
fac.L
```

```
▸[3, 2, 1]
```

```
factorise_lu_pivot(AA)[3]  # p
```

```
3×3 LowerTriangular{Float64, Matrix{Float64}}:
 1.0        ⋅    ⋅
 0.285714  1.0   ⋅
 0.142857  0.5  1.0
```

```
factorise_lu_pivot(AA)[1]  # L
```

⚠ TODO ⚠

Explain how to disable pivoting in julia and how to get the permuation matrix and so on

```
TODO("Explain how to disable pivoting in julia and how to get the permuation matrix and so on")
```

```
MM = 3×3 Matrix{Int64}:
      2  3   -4
      2  2   -2
      6  4  -11
```

```
MM = [2 3  -4;
      2 2  -2;
      6 4 -11]
```

```
3×3 Matrix{Float64}:
 0.0  0.0  1.0
 1.0  0.0  0.0
 0.0  1.0  0.0
```

```
lu(MM).P
```

```
3×3 adjoint(::Matrix{Int64}) with eltype Int64:
 0  0  1
 1  0  0
 0  1  0
```

```
([0 0 1; 0 1 0; 1 0 0] * [1 0 0; 0 0 1; 0 1 0])'
```

```
LinearAlgebra.LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0  0.0  0.0
 1.0  1.0  0.0
 3.0  5.0  1.0
U factor:
3×3 Matrix{Float64}:
 2.0   3.0  -4.0
 0.0  -1.0   2.0
 0.0   0.0  -9.0
```

```
lu(MM, NoPivot())
```

```
LinearAlgebra.LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0       0.0  0.0
 0.333333  1.0  0.0
 0.333333  0.4  1.0
U factor:
3×3 Matrix{Float64}:
 6.0  4.0      -11.0
 0.0  1.66667   -0.333333
 0.0  0.0        1.8
```

```julia
lu(MM)
```

Memory usage and fill-in

Sparse matrices

The matrices and linear systems one encounters in physics and engineering applications not infrequently reach huge sizes. For example in the numerical solution of partial differential equations using finite difference or finite element methods the vector of unknowns frequently has a dimension in the order of $n \simeq 10^6$. This implies that matrices, i.e. entities from $\mathbb{R}^{n \times n}$ could hold around 10^{12} elements.

In double precision, i.e. 64-bit numbers, each entry requires 8 bytes. As a result storing all elements of such a matrix explicitly in memory reqires

```
7.275957614183426
```

```julia
10^12 * 8 / 1024^4  # TiB
```

tibibyes of storage. This is a challenge even for standard compute clusters, where typically a node has around 512 GiB. Laptops nowadays feature around 32 GiB, so would be completely out of the question to solve such problems.

However, in many applications the arising matrices are sparse, meaning that they contain a large number of zero elements, which do not need to be stored. Let us discuss a few examples.

Definition: Full matrix

A matrix $A \in \mathbb{R}^{n \times n}$ is called **full** if the number of non-zero elements is at the order of n^2. For such matrices almost all elements are non-zero and need to be stored.

Full matrices are the "standard case" and for them memory constraints usually set the limit of the size of problems, which can be tackled. For example an 32 GiB memory laptop can store around $4 \cdot 10^9$ double-precision numbers, which means that linear problems with more than around $n \simeq 60000$ unknows cannot be treated.

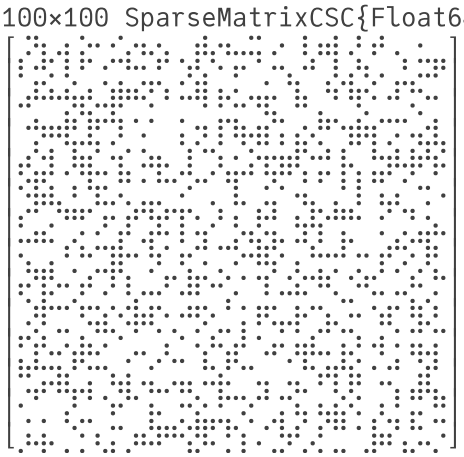
Definition: Sparse matrix

A matrix $A \in \mathbb{R}^{n \times n}$ is called **sparse** if the number of non-zero elements is at the order of n.

If we know which elements are zero, we can thus save memory by only storing those elements, which are non-zero. For this purpose the `SparseArrays` Julia package implements many primitives for working with sparse arrays.

This includes, generating random sparse arrays:

```
Asp = 100×100 SparseMatrixCSC{Float64, Int64} with 1500 stored entries:
```



```
Asp = sprand(100, 100, 0.15)
```

or simply sparsifying a dense array using the `sparse` function, which converts a full matrix to a sparse matrix by dropping the explicit storage of all zero entries.

```
M = 4×4 Matrix{Int64}:
     1  0  0   5
    -2  0  1   0
     0  0  6   0
     0  1  0  -1
```

```
M = [ 1 0 0  5;
     -2 0 1  0;
      0 0 6  0;
      0 1 0 -1]
```

```
4×4 SparseMatrixCSC{Int64, Int64} with 7 stored entries:
  1  ⋅  ⋅   5
 -2  ⋅  1   ⋅
  ⋅  ⋅  6   ⋅
  ⋅  1  ⋅  -1
```

```
sparse(M)
```

Using the `SparseArray` data structure from `SparseArrays` consistently allows to fully exploit sparsity when solving a problem. As a result the storage costs scale only as $O(n)$. With our laptop of 32 GiB memory we can thus tackle problems with around $n \simeq 4 \cdot 10^9$ unknowns — much better than the 60000 we found when using full matrices.

Finally we introduce a special kind of sparse matrix:

Definition: Band matrix

A matrix $A \in \mathbb{R}^{n \times n}$ is called a **band matrix** with bandwidth d if $A_{ij} = 0$ when $|j - i| > d$. Every line of the matrix contains at most $2d + 1$ non-zero elements and the number of non-zeros thus scales as $O(nd)$.

An example for a banded matrix with bandwidth 5 is:

```
band = 105×105 SparseMatrixCSC{Float64, Int64} with 817 stored entries:
```

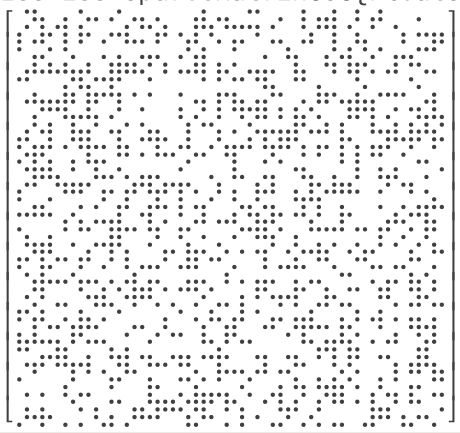
LU factorisation of sparse matrices

Since the amount of available memory can put hard constraints on the size of linear systems which can be solved, we now want to investigate the memory requirement of LU factorisation $\mathbf{A} = \mathbf{L}\mathbf{U}$.

If $\mathbf{A}$ is a full matrix, then we know that $\mathbf{L}$ and $\mathbf{U}$ are triangular, thus contain less non-zero elements than $\mathbf{A}$ itself.

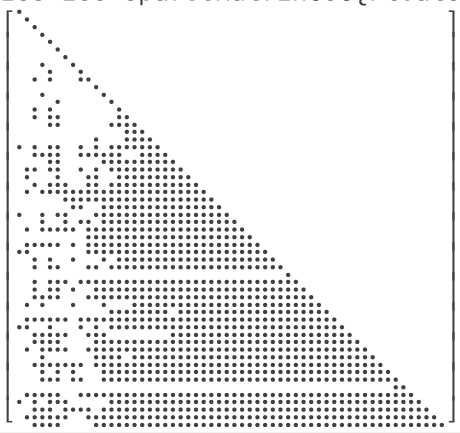
Let's contrast this with the sparse matrices we have considered above. First the random sparse matrix:

```
100×100 SparseMatrixCSC{Float64, Int64} with 1500 stored entries:
```



```
Asp
```

```
100×100 SparseMatrixCSC{Float64, Int64} with 2957 stored entries:
```



```
lu(Asp).L  # U looks similar
```

Storing both L and U thus requires a total number of

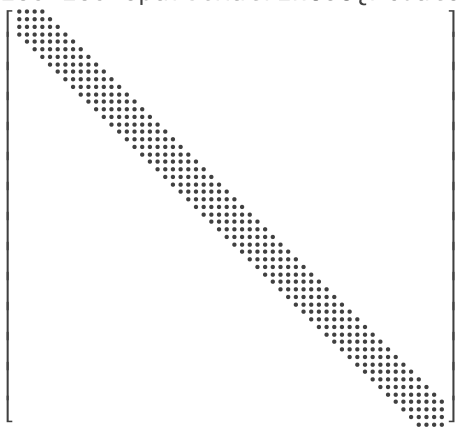
```
5699
```

```
nnz(lu(Asp).L) + nnz(lu(Asp).U)
```

non-zero elements, which is about **4 times as much** as the original matrix !

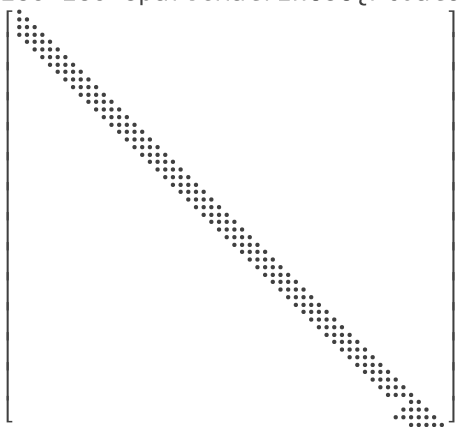
Now the banded matrix:

```
105×105 SparseMatrixCSC{Float64, Int64} with 817 stored entries:
```



```
band
```

```
105×105 SparseMatrixCSC{Float64, Int64} with 611 stored entries:
```



```
lu(band).L  # U looks similar
```

At least the banded structure is kept, but still we require

```
1222
```

```
nnz(lu(band).L) + nnz(lu(band).U)
```

nonzeros, which is about 1.5 times than the original in this case.

In both cases storing the $\mathbf{L}$ and the $\mathbf{U}$ factors require more non-zero elements than the original matrix. This phenomenon is usually referred to as **fill in**.

In particular for the sparse matrix `Asp` LU factorisation did not preserve the structure at all. In contrast it almost resulted in a full matrix in the lower right corner of the $\mathbf{L}$. For $\mathbf{U}$ this looks exactly the same. In fact one can show that in general even for sparse matrices the **memory usage of LU factorisation** is $O(n^2)$. Therefore while one may be able to store a sparse matrix $\mathbf{A}$ with a huge size in memory (due to the $O(n)$ memory cost), one may actually run out of memory while computing the factorisation.

Let us add that for banded matrices the situation is slightly better as one can show that in this case the fill-in takes place at most inside the band. As a result the memory requirement stays at $O(nd)$.

Computational cost of LU factorisation

In this section we want to investigate how long it will take a computer to perform LU factorisation on a large matrix.

Modern laptop computers nowadays have a clock frequence fo a few Gigahertz (GHz). This means they are able to perform about 10^9 operations per second, where for simplicity we assume that "one operation" is an elementary addition, multiplication, division and so on. If we can determine how many such operations are needed to perform LU factorisation we can estimate the computational cost.

Before we look at the LU algorithm (Algorithm 4) we first understand a few simpler cases from linear algebra:

Scalar product

Given two vectors $x, y \in \mathbb{R}^n$ consider computing the scalar product

$$x \cdot y = x^T y = \sum_{i=1}^{n} x_i \, y_i$$

which in code is achieved as

```
scalar_product (generic function with 1 method)
```

```julia
function scalar_product(x, y)
    result = 0.0
    for i in 1:length(x)
        result = result + x[i] * y[i]
    end
    result
end
```

In the loop for each iteration we require 1 multiplication and 1 addition. In total the `scalar_product` function therefore requires n multiplications and n additions. The number of elementary operations is thus $2n$. We say the computational cost is $O(n)$ (i.e. on the order of n or linear in n).

If we take the dimensionality $n = 1000$ the number of operations is $O(1000)$. On a 1 GHz computer (with 10^9 operations per second) this therefore takes about 10^{-6} seconds ... hardly noticable.

Matrix-vector product

Given a matrix $A \in \mathbb{R}^{n \times n}$ and a vector $x \in \mathbb{R}^n$ the matrix-vector product $y = Ax$ is computed as

$$y_i = \sum_{j=1}^{n} A_{ij} x_j \qquad \text{for } i = 1, \ldots, n$$

or in code

```
matrix_vector_product (generic function with 1 method)
```

```julia
function matrix_vector_product(A, x)
    y = zeros(size(A, 1))  # Create a vector of zeros (#)

    for i in 1:size(A, 1)
        for j in 1:size(A, 2)
            y[i] = y[i] + A[i, j] * y[j]  # (*)
        end
    end
    result
end
```

In the innermost loop we observe again that we require 1 addition and 1 multiplication per iteration. This instruction is performed once for each combination of i and j, so we look at the limits of each of the nested loops. The loop over i runs over n values (the number of rows of A) and the loop over j over n values as well (the number of columns of A). In total the inner most instruction $(*)$ is thus run n^2 times, each times costing 2 operations. The total cost is thus $O(n^2)$.

For our case with $n = 1000$ and a 1 GHZ computer we thus now need $(10^3)^2/10^9 s = 10^{-3} s = 1ms$, which is again a rather short time.

▶ **Expert information: What about the allocation in `(#)` ?**

LU factorisation

We revisit Algorithm 4:

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$

Output: $\mathbf{U} \in \mathbb{R}^{n\times n}$, $\mathbf{L} \in \mathbb{R}^{n\times n}$

- $\mathbf{A}^{(1)} = \mathbf{A}$
- for $k = 1, \ldots, n-1$ *(algorithm steps)*
  - $L_{kk} = 1$
  - for $i = k+1, \ldots, n$ *(Loop over rows)*
    - $L_{ik} = \frac{A_{ik}^{(k)}}{A_{kk}^{(k)}}$
    - for $j = k+1, \ldots n$ *(Loop over columns)*
      - $A_{ij}^{(k+1)} = A_{ij}^{(k)} - L_{ik}A_{kj}^{(k)}$
- $\mathbf{U} = \mathbf{A}^{(n)}$

We notice that the instruction at the innermost loop level is

$$A_{ij}^{(k+1)} = A_{ij}^{(k)} - L_{ik}A_{kj}^{(k)},$$

which costs again 1 addition and 1 multiplication and which is executed once for each tuple (i, j, k). In the worst case k is takes $n-1$ elements, i also $n-1$ elements (assume $k = 1$) and j again in its worst case takes $n-1$ elements. In total the cost is thus $(n-1)^3 = O(n^3)$.

For our case with $n = 1000$ and a 1 GHZ computer the computation thus needs approximately $(10^3)^3/10^9 s = 1s$, which starts to be a noticable amount of time. If we even consider $n = 10^5$ all of a sudden it takes $10^3 s$, which is about 15 minutes. For large matrices the cost of computing LU factorisation can thus become important !

For **banded matrices** the computational scaling is a little improved. E.g. consider LU factorisation on a banded matrix A with band width d. Then bothe the loop over i (row loop) as well as the loop over j (column loop) in the LU factorisation algorithm can be truncated and at most run over d elements. As a result the computational cost at worst becomes $O(nd^2)$ which for a small band width (i.e. $d < n$) is substantially less than $O(n^3)$.

Overview of LU factorisation computational cost and memory cost

We summarise the cost of LU factorisation in a table:

type of $n \times n$ matrix	computational cost	memory usage
`full matrix`	$O(n^3)$	$O(n^2)$
`banded matrix, band width` d	$O(n\,d^2)$	$O(n\,d)$

Numerical stability

To close off our discussion of Gaussian elimination we consider the question of its numerical stability, i.e. how it is effected by the round off errors introduced in the algorithm by finite-precision floating-point arithmetic.

In this chapter our goal has been to solve a linear system of the form

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \tag{5}$$

where we are given a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ as well as a right-hand side $\mathbf{b} \in \mathbb{R}^n$ and we wish to solve for $\mathbf{x}$ We will later refer to this system as the **exact system**.

Since the computer in general is unable to represent the matrix and right-hand side exactly in finite-precision floating point arithmetic, even providing this problem to a computer is already associated with making a small error: in practice the computer is only able to obtain a solution $\widetilde{\mathbf{x}}$ to a slighly modified linear system

$$\widetilde{\mathbf{A}}\widetilde{\mathbf{x}} = \widetilde{\mathbf{b}}, \tag{6}$$

where $\widetilde{\mathbf{A}}$ is an approximation to $\mathbf{A}$, $\widetilde{\mathbf{b}}$ is an approximation to $\mathbf{b}$. This linear system we will refer to as the **perturbed system**.

In standard floating-point arithmetic the *relative error* in representing any number is around 10^{-16}. Mathematically we can find the following relationships between the elements of $\widetilde{\mathbf{b}}$ and $\mathbf{b}$ as well as $\widetilde{\mathbf{A}}$ and $\mathbf{A}$, respectively:

$$\begin{aligned} \tilde{b}_i &= b_i\,(1 + \varepsilon_i) && \textbf{where } \varepsilon_i \textbf{ is on the order of } 10^{-16} \\ \widetilde{A}_{ij} &= A_{ij}\,(1 + \eta_{ij}) && \textbf{where } \eta_{ij} \textbf{ is on the order of } 10^{-16} \end{aligned}$$

To understand the numerical stability when solving linear systems our goal is now to understand how far the solution $\widetilde{\mathbf{x}}$ — obtained on a computer — differs from $\mathbf{x}$ — the solution to (5), the true linear system we want to solve.

To simplify matters in this course we will not attempt to answer this question in full generality, but concentrate on the case $\eta_{ij} = 0$, i.e. a setting where we only consider the rounding-error in the right-hand side $\widetilde{\mathbf{b}}$ and $\mathbf{b}$, but assume that the system matrix is accessible to the computer without error ($\widetilde{\mathbf{A}} = \mathbf{A}$). For an alternative and more detailed discussion see also chapter 2.8 of Driscoll, Brown: *Fundamentals of Numerical Computation*.

To provide a motivation why this type of analysis matters in practice, we first consider the following example:

Example: Rounding error in a 2x2 system

Consider the *exact* linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b} \qquad \Leftrightarrow \qquad \begin{pmatrix} 1 & 10^{-16} \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

with solution $(x_1, x_2) = (1, 0)$ and the *perturbed* linear system

$$\mathbf{A}\widetilde{\mathbf{x}} = \widetilde{\mathbf{b}} \qquad \Leftrightarrow \qquad \begin{pmatrix} 1 & 10^{-16} \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \widetilde{x}_1 \\ \widetilde{x}_2 \end{pmatrix} = \begin{pmatrix} 1 + 10^{-16} \\ 1 \end{pmatrix},$$

where only the first component of the right-hand side has been perturbed by 10^{-16}. This second system has solution $(\widetilde{x}_1, \widetilde{x}_2) = (1, 1)$. The second component of this solution is thus *completely wrong*!

The small perturbation of 10^{-16}, which may readily occurr just by inputing the data to a computer, can already have an impactful effect on the solution and in fact render the solution we obtain from our our computation (i.e. $\widetilde{\mathbf{x}}$) rather far from the answer we are actually after (i.e. $\mathbf{x}$).

Matrix norm and condition numbers

To analyse the error $\mathbf{x} - \widetilde{\mathbf{x}}$ mathematically, we first have to introduce some notation.

Recall that for a vector $\mathbf{v} \in \mathbb{R}^n$ its Euclidean norm is $\|\mathbf{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$.

We define the following:

Definition: Relative error of linear systems

The **relative error** in $\widetilde{\mathbf{x}}$, the solution to the perturbed system (6), is the quantity

$$\frac{\|\mathbf{x} - \widetilde{\mathbf{x}}\|}{\|\mathbf{x}\|},$$

where $\mathbf{x}$ is the exact solution, i.e. the solution to the exact linear system (5) one actually wanted to solve.

We further need a generalisation of norms to matrices:

Definition: Matrix norm

Given $\mathbf{M} \in \mathbb{R}^{m \times n}$ a real matrix (not necessarily square), we define the matrix norm of $\mathbf{M}$ as

$$\|\mathbf{M}\| = \max_{\substack{\mathbf{v} \in \mathbb{R}^n \\ \mathbf{v} \neq \mathbf{0}}} \frac{\|\mathbf{M}\mathbf{v}\|}{\|\mathbf{v}\|}.$$

The previous definition implies in particular that

$$\|\mathbf{M}\mathbf{v}\| \leq \|\mathbf{M}\| \, \|\mathbf{v}\| \quad \forall \mathbf{x} \in \mathbb{R}^n. \tag{7}$$

With this definitions in place we begin our analysis for (5) and (6) where $\mathbf{A} \in \mathbb{R}^{n \times n}$. From the definition of the exact and perturbed linear systems

$$\mathbf{A}\mathbf{x} = \mathbf{b} \qquad \text{and} \qquad \mathbf{A}\widetilde{\mathbf{x}} = \widetilde{\mathbf{b}}$$

we obtain by subtraction and since $\mathbf{A}$ is invertible

$$\mathbf{A}\left(\mathbf{x} - \widetilde{\mathbf{x}}\right) = \mathbf{b} - \widetilde{\mathbf{b}} \qquad \Rightarrow \qquad \mathbf{x} - \widetilde{\mathbf{x}} = \mathbf{A}^{-1}\left(\mathbf{b} - \widetilde{\mathbf{b}}\right)$$

Using (7) we therefore have

$$\|\mathbf{x} - \widetilde{\mathbf{x}}\| \leq \left\|\mathbf{A}^{-1}\right\| \left\|\mathbf{b} - \widetilde{\mathbf{b}}\right\|.$$

Furthermore we have from applying (7) to $\mathbf{A}\mathbf{x} = \mathbf{b}$:

$$\|\mathbf{b}\| \leq \|\mathbf{A}\| \, \|\mathbf{x}\| \qquad \Rightarrow \qquad \frac{1}{\|\mathbf{x}\|} \leq \frac{\|\mathbf{A}\|}{\|\mathbf{b}\|}$$

such that with the above result we can bound the relative error as

$$\frac{\|\mathbf{x} - \widetilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \left\|\mathbf{A}^{-1}\right\| \left\|\mathbf{b} - \widetilde{\mathbf{b}}\right\| \cdot \frac{\|\mathbf{A}\|}{\|\mathbf{b}\|} = \underbrace{\left\|\mathbf{A}^{-1}\right\| \, \|\mathbf{A}\|}_{=\kappa(\mathbf{A})} \frac{\left\|\mathbf{b} - \widetilde{\mathbf{b}}\right\|}{\|\mathbf{b}\|}. \tag{8}$$

From as stability analysis point of view the quantity $\kappa(\mathbf{A}) = \left\|\mathbf{A}^{-1}\right\| \, \|\mathbf{A}\|$ tus relates the relative error in the right-hand side (input quantity) to the relative error in the solution $\mathbf{x}$ (output quantity). It is thus the condition number for a linear system:

Definition: Condition number of a linear system

Given a linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ with an invertible system matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ its **condition number** is defined as

$$\kappa(\mathbf{A}) = \left\|\mathbf{A}^{-1}\right\| \, \|\mathbf{A}\|.$$

Due to the importance of solving linear systems in linear algebra one usually calls $\kappa(\mathbf{A})$ also the **condition number of the matrix** $\mathbf{A}$.

But notably (8) can be interpreted even more generally as the way it the solution of linear systems are changed as we change the right-hand side as is summarised below:

Theorem 2: Change of solution when changing the right-hand side

Given a linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$, which we solve for $\mathbf{x}$ and a related linear system $\mathbf{A}\widetilde{\mathbf{x}} = \widetilde{\mathbf{b}}$ which we solve for $\widetilde{\mathbf{x}}$, then the two solutions are related as

$$\frac{\|\mathbf{x} - \widetilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \, \frac{\left\|\mathbf{b} - \widetilde{\mathbf{b}}\right\|}{\|\mathbf{b}\|}.$$

Before we continue to develop expression (8) we pause for a second to discuss how condition numbers can be computed in practice.

First let us introduce the following notation:

Definition: Minimal and maximal eigenvalues

Given a diagonalisable matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ we denote by $\lambda_{\max}(\mathbf{M})$ the *largest* eigenvalue of $\mathbf{M}$ and by $\lambda_{\min}(\mathbf{M})$ the *smallest* eigenvalue of $\mathbf{M}$, i.e.

$$\lambda_{\max}(\mathbf{M}) = \max_{i=1,\ldots,n} \lambda_i(\mathbf{M}) \qquad \text{and} \qquad \lambda_{\min}(\mathbf{M}) = \min_{i=1,\ldots,n} \lambda_i(\mathbf{M})$$

where $\lambda_i(\mathbf{M})$ for $i = 1, \ldots, n$ are the eigenvalues of $\mathbf{M}$.

It turns out that the minimal and maximal eigenvalues provide a convenient way to compute matrix norms:

Lemma 3: Computing matrix norms

For any matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$

$$\|\mathbf{M}\| = \sqrt{\lambda_{\max}(\mathbf{M}^T\mathbf{M})}.$$

If $\mathbf{M}$ is moreover square and invertible, then

$$\|\mathbf{M}^{-1}\| = \sqrt{\lambda_{\max}(\mathbf{M}^{-T}\mathbf{M}^{-1})} = \frac{1}{\sqrt{\lambda_{\min}(\mathbf{M}^T\mathbf{M})}}.$$

Based on this we can deduce a few useful formulas for computing condition numbers:

- For any invertible $\mathbf{A} \in \mathbb{R}^{n \times n}$ the condition number can be computed as

$$\kappa(\mathbf{A}) = \left\|\mathbf{A}^{-1}\right\| \, \|\mathbf{A}\| = \frac{\sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}}{\sqrt{\lambda_{\min}(\mathbf{A}^T\mathbf{A})}}.$$

- If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is invertible and **symmetric** then $\mathbf{A}^T = \mathbf{A}$. As a result we observe that

$$\lambda_i(\mathbf{A}^T\mathbf{A}) = \lambda_i(\mathbf{A}^2) = \lambda_i(\mathbf{A})^2 \qquad \text{for all } i = 1, \ldots, n.$$

  As a result

$$\kappa(\mathbf{A}) = \frac{\sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}}{\sqrt{\lambda_{\min}(\mathbf{A}^T\mathbf{A})}} = \frac{\sqrt{\max_{i=1,\ldots,n} \lambda_i(\mathbf{A})^2}}{\sqrt{\min_{i=1,\ldots,n} \lambda_i(\mathbf{A})^2}} = \frac{\max_{i=1,\ldots,n} |\lambda_i(\mathbf{A})|}{\min_{i=1,\ldots,n} |\lambda_i(\mathbf{A})|}$$

- If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is **both symmetric and positive definite** (i.e. all eigenvalues are strictly positive), then we can drop the moduli in the above expression and obtain:

Lemma 4: Condition number of symmetric, positive-definite matrices

If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is symmetric and positive definite, then

$$\kappa(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})}$$

Rounding error in a 2x2 system (continued)

We continue our example from earlier and evaluate the condition number of the matrix

$$\mathbf{A} = \begin{pmatrix} 1 & 10^{-16} \\ 1 & 0 \end{pmatrix}.$$

This matrix is not symmetric and we therefore use the expression

$$\kappa(\mathbf{A}) = \frac{\sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}}{\sqrt{\lambda_{\min}(\mathbf{A}^T\mathbf{A})}},$$

which requires us to compute the eigenvalues of $\mathbf{A}^T\mathbf{A}$. We note

$$\mathbf{A}^T\mathbf{A}) = \begin{pmatrix} 1 & 1 \\ 10^{-16} & 0 \end{pmatrix} \begin{pmatrix} 1 & 10^{-16} \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 10^{-16} \\ 10^{-16} & 10^{-32} \end{pmatrix}$$

and

$$\det(\mathbf{A}^T\mathbf{A}) - \lambda I) = \lambda^2 - (2 + 10^{-32})\lambda + 10^{-32},$$

such that the two eigenvalues of $\mathbf{A}^T\mathbf{A})$ are

$$\lambda_{\pm}(\mathbf{A}^T\mathbf{A})) = \frac{1}{2}\left(2 + 10^{-32} \pm \sqrt{4 + 10^{-64}}\right).$$

We conclude that

$$\lambda_{\max}(\mathbf{A}^T\mathbf{A}) \approx 2 \qquad \text{and} \qquad \lambda_{\min}(\mathbf{A}^T\mathbf{A}) \approx \frac{10^{-32}}{2}$$

therefore

$$\kappa(\mathbf{A}) \approx 2 \cdot 10^{16},$$

which is huge !

In Julia code the condition number of a matrix is computed programmatically using the `cond` function. It automatically makes a guess to use the most appropriate of teh above formulas. For the above example we can compute:

```
2.0000000000000004e16
```

```julia
let
    A = [1 1e-16;
         1  0    ]
    κ = cond(A)
end
```

Stability result

With these computational tools let us return to the previously obtained relationship (8) between relative error in the right-hand side and relative error in the solution of the exact and perturbed systems:

$$\frac{\|\mathbf{x}-\widetilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A})\,\frac{\left\|\mathbf{b}-\widetilde{\mathbf{b}}\right\|}{\|\mathbf{b}\|}.$$

In our setup we said that storing $\mathbf{b}$ on the computer (which leads to $\widetilde{\mathbf{b}}$) introduces a small relative error at each of the elements of the right-hand side, leading to a relationship

$$\tilde{b}_i = b_i\,(1+\varepsilon_i).$$

Let us further define

$$\epsilon = \max_{i=1,\ldots,n} |\varepsilon_i|.$$

For standard double-precision floating-point number as discusset at the beginning of this section, we have that $\epsilon \approx 10^{-16}$.

With this setup we can simplify the expression of the absolute error of right-hand side:

$$\left\|\mathbf{b}-\widetilde{\mathbf{b}}\right\| = \sqrt{\sum_{i=1}^{n} b_i^2 \varepsilon_i^2} \leq \sqrt{\sum_{i=1}^{n} b_i^2 \underbrace{\max_{i=1,\ldots,n} |\varepsilon_i|^2}_{=\epsilon^2}} = \epsilon\,\|\mathbf{b}\|.$$

This leads to the following stability result:

Theorem 5: Stability of solving linear systems

Given a linear system $\mathbf{A}\mathbf{x}=\mathbf{b}$ and $\widetilde{\mathbf{x}}$ the solution to a perturbed linear system $\mathbf{A}\widetilde{\mathbf{x}}=\widetilde{\mathbf{b}}$, where $\tilde{b}_i = b_i\,(1+\varepsilon_i)$ for $i=1,\ldots,n$ and $\epsilon = \max_{i=1,\ldots,n} |\varepsilon_i|$, then

$$\frac{\|\mathbf{x}-\widetilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A})\,\epsilon$$

This inequality shows that the condition number of the matrix $\mathbf{A}$ plays the role of *amplifying* the round-off error introduced by the floating-point representation of the right-hand side.

Rounding error in a 2x2 system (continued)

Using Theorem 5 we can finally understand the behaviour we observe in our example. Previously we found that a small perturbation $1 \to 1 + 10^{-16}$ in one of the elements of the right hand side, introduces a change in the solution from $(x_1, x_2) = (1, 0)$ to $(\tilde{x}_1, \tilde{x}_2) = (1, 1)$. This is a relative error of

$$\frac{\|\mathbf{x} - \tilde{\mathbf{x}}\|}{\|\mathbf{x}\|} = \frac{1}{1} = 1.$$

Note that in this example $\epsilon \approx 10^{-16}$ just like for the case of a rounding error in the right-hand side.

As we found previously

$$\kappa(\mathbf{A}) \approx 2 \cdot 10^{16},$$

such that this large relative error is explained by the large condition number of this matrix.

Numerical analysis

1. Introduction
2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Interpolation
6. Direct methods for linear systems
7. Iterative methods for linear systems
8. Eigenvalue problems
9. Numerical integration
10. Numerical differentiation
11. Initial value problems
12. Boundary value problems